

Analysis: Pony Express

Pony Express: Analysis

Small dataset

In the Small dataset, we are given a line of cities and the distances between them, and we want to go from the first to the last city in the line. However, we need to minimize the time and not the distance, and the obvious way to transform between the two (divide by speed) does not work directly, because the horses we can use have different speeds. The solution, as in many minimization algorithms on a path that only moves forward, is to use [dynamic programming](#).

Let us define $f(i)$ as the quickest way to get from city i to the finish line, assuming we start using the horse at city i . Of course, $f(1)$ is the answer to the only query.

We do not know where we need to change horses, but we can try every possible intermediate city j to change horses, thus completely defining $f(i)$ by:

- $f(i) = \min \{ \text{distance}(i, j) / \text{speed}(i) + f(j) : \text{for all } j \text{ such that } \text{distance}(i, j) \leq \text{endurance}(i) \}$, for $i < N$.
- $f(N) = 0$.

This solves the problem in $O(N^2)$ because the domain of f is of size N and the iteration to calculate each value of the domain takes time $O(N)$ if we memoize the results. There are other dynamic programming approaches that also work in the same amount of time.

Large dataset

In the Large dataset, we are given a graph G of cities with distances between them instead of just a line, but the setup of the problem is otherwise the same.

As mentioned above, one relatively unusual feature of this graph problem is that the desired result (time) is not in the same units as the weights of the graph's edges (distance). Moreover, the obvious way to transform between the two (speed) is not a constant. This observation leads to a key insight: we should construct a new graph with weights of time instead of distance. That way, we can apply a [shortest path](#) algorithm to get the result we want.

We can do this by defining a graph G' that has the same nodes as G , but in which the weight of edge (i, j) is the time that it takes to go from city i to city j . As we said above, there is no fixed speed, but we can fix it. The edge (i, j) then represents the time it takes to go from city i to city j using a single horse, and the obvious choice is to use the horse at the departure city i . Let us call that horse h . In this way, the edge (i, j) on G' represents a path on G , traversed using a single horse. That means the edge (i, j) exists if and only if there is a path between i and j in G with distance less than or equal to h 's endurance. Moreover, the weight (i, j) is a time now defined by a single speed, h 's speed, and thus it is just the distance between i and j in G , divided by h 's speed. You can see that a minimum path from a to b in G' represents a succession of edges in G' , that is, a succession of single-horse paths in G , which is exactly what a solution looks like!

Since the limits are low enough, and we need all shortest paths in G to construct G' , and many shortest paths in G' to answer many queries, the best option is to use an all-pairs shortest path algorithm. [Floyd-Warshall](#) is the easiest and fastest to implement, but others would work too. To

also check for existence of paths, you can use the trick of setting weights to "infinity", that is, a distance too large for an actual shortest path (larger than maximum distance $\times$ number of total edges). Thus, an infinity distance simply means the edge or path does not exist in the graph.

To summarize, this pseudocode solves the problem:

1. Apply Floyd-Warshall to input G getting distances between all pairs of nodes.
2. Create G' by adding all edges (i, j) such that the distance between i and j in G is less than or equal to the horse starting at city i 's endurance, and set their weights to that same distance divided by that horse's speed.
3. Apply Floyd-Warshall to G' to get minimum times between all pairs of nodes.
4. Read queries and answer immediately from the output of the last step.

This solution takes time $O(N^3)$. Both uses of Floyd-Warshall take time $O(N^3)$, and creating G' as an adjacency matrix only takes time $O(N^2)$.

Analysis: Stable Neigh-bors

Stable Neigh-bors: Analysis

Small dataset

In the Small dataset, all of the unicorns have manes with only a single primary color: red, yellow, or blue. None of these manes share any primary colors, so our only restriction is that two unicorns with the same mane color cannot be next to each other. However, one color might be so common that it is impossible to avoid putting two unicorns with that color next to each other. If we space unicorns with the most common color out, with some other unicorn between each pair of them, then we can accommodate up to $\text{floor}(N/2)$ of the most common color. If we have more than that number with any color, the case is impossible.

Otherwise, we can extend this "every other stall" strategy to work for any case. Start at some arbitrary position on the ring of stalls, and place unicorns in the first, third, fifth, etc. stalls, all the way around the ring. When you reach the starting point again, continue to place unicorns in the second, fourth, sixth, etc. stalls. As you do this, place all of the unicorns with the most common mane color, and then all the unicorns with the next most common mane color, and then all the rest. Because all of these colors have a frequency no greater than $\text{floor}(N/2)$, it is not possible for this placement strategy to put two unicorns with the same color next to each other.

Large dataset

The Large dataset introduces more complications. Unicorns with a secondary-colored mane (orange, green, or violet) can only have one type of neighbor. An orange-maned unicorn must be next to two blue-maned unicorns; similarly, a yellow must be next to purples, and a red must be next to greens.

How many instances of a primary color do we need to "surround" all unicorns with the corresponding secondary-colored mane? If the secondary color and its primary neighbor are the only two colors available, then there must be equal numbers of those two colors. Otherwise, if there are S instances of the secondary color, we need at least $S + 1$ instances of the primary color.

Moreover, we might as well put all secondary colors of the same type together (separated by the primary color, of course) in a single chain. Any valid arrangement without that property can be rearranged to have that property. For instance, suppose that we have two separate R-G-R chains separated by two other valid chains Z and Z' : R-G-R-Z-R-G-R-Z'. We can just rearrange this to R-G-R-G-R-Z-R-Z', which must also be valid.

A final useful insight is that an R-G-R-G-R chain, for example, acts just like a single R in terms of what can neighbor it on either side. So one strategy for the Large is: first, check for the case mentioned above in which only one secondary color and its neighboring primary color are present. Otherwise, for each secondary color, check that there are enough primary-color unicorns to surround the secondary-color unicorns in a chain. Then pretend that each of these chains is just a single instance of the primary-colored mane from that chain; this reduces the problem to primary colors, and the algorithm for the Small dataset works. You can substitute each chain back in (for some arbitrary instance of the appropriate primary color) once that algorithm has finished.

These Small and Large solutions run in $O(N)$ time, and they are bound by reading the input, checking color frequencies, and printing the output. Other, more complex solutions (e.g., dynamic programming) also exist.

Analysis: Steed 2: Cruise Control

Pop quiz, hotshots! This problem seems pretty complicated at first glance. What do you do?

One natural strategy is to try binary searching on Annie's speed, but it is difficult to directly determine whether a given speed avoids passing another horse; the input data alone does not tell us where each horse is at any given time, because horses might slow other horses down. In theory, we could figure out when faster horses catch up to slower horses and slow down, determine the exact path of each horse, and check whether our chosen speed crosses any of those paths. With only up to two horses in test set 1, this sort of calculation is feasible, but it would be laborious for test set 2.

However, we can avoid all of that work via some observations. To maximize cruising speed, Annie's horse should reach the destination at exactly the same time as the horse ahead of her (let's call it Horse A); there is no reason to leave a gap. Either Horse A will reach the destination without having to slow down (and so it will be the one that directly limits Annie's speed), or it will be slowed down at some point by the horse ahead of it (let's call it Horse B). The same is true for Horse B: either it will never have to slow down (and so it will be the one that ultimately limits Annie's speed), or it will be slowed down by the horse ahead of *it*, and so on. So there will be a single "limiting horse" on the road that ultimately determines how fast Annie's horse can reach the destination. We claim that this "limiting horse" is the only horse that matters, and we can disregard all of the others!

It is easy to see that we can ignore the horses to the east of the limiting horse; they will reach and pass the destination before the limiting horse gets there. What about the "intermediate horses" between Annie and the limiting horse? We know from the way we have defined the limiting horse that every intermediate horse will catch up to the limiting horse before reaching the destination. (If one did not, then *it* would be the limiting horse.) Suppose that Annie chooses a cruising speed that gets her to the destination at exactly the same time as the limiting horse. We certainly cannot go faster than this. Moreover, this speed is safe: it cannot possibly cause Annie to pass any of the intermediate horses. If she were going fast enough to overtake an intermediate horse, then she would definitely be going fast enough to pass the limiting horse, since every intermediate horse will catch up to the limiting horse. This would cause a contradiction. Therefore, we do not need to worry about the intermediate horses or their interactions with each other.

So, once we have identified the limiting horse, the strategy is simple: go at the exact speed that will cause Annie to reach the destination at the same time as the limiting horse. This speed can be found in constant time. We could identify the limiting horse directly via the argument in our third paragraph above, but even this would be unnecessary work. Instead, for each horse in turn, we can pretend that it is the limiting horse and calculate the cruising speed that it would force. Then the smallest of those speeds is our answer. (If any horse allows a faster cruising speed than another, it cannot be the limiting horse, because that cruising speed would cause Annie to pass the true limiting horse.) This takes $O(N)$ time.